Learning Choir: a Teaching Series in Architecture, Platform and Security

A map of six documents, written to teach the owner the system he is building. 14 October 2026. Repository: `origin/main` at 980ffaf4e. Mutation class: **green** (docs only).

The owner, today: working at the limit and wanting to learn faster in software architecture, platform engineering and security. This series is built for that. Each document takes one part of Choir and walks the same arc that the computer isolation security review walks:

1. **Primer.** The general idea, from first principles, as you would meet it in any well-run platform team: what an append-only log is, what a circuit breaker is, what a snapshot is.
2. **How Choir does it.** The same idea in this repository, with file paths and function names, so you can open the code and check.
3. **Where it is weak.** The real incidents from `docs/problems/` and today's reports where the idea broke.
4. **Options and trade-offs.** What the alternatives are and what each one costs.
5. **Glossary, five self-check questions, and the three files to read next.**

Every claim is grounded in code or a dated Choir document. Each claim carries one of four evidence labels: **code** (read in the repository at 980ffaf4e, with a path), **doc** (an existing Choir document), **upstream** (a vendor or project document, cited), and **inference** (reasoning that has not been tested). Nothing in the series was read from a live host; where a number comes from Node B it comes from a dated report that measured it.

The six documents

#	document	what it teaches	the one idea to keep
1	The ledger and the store (PDF)	append-only logs and hash chains, compare-and-swap, event sourcing, commitment records, Dolt (prolly trees, commits, `AS OF`, branches), the object graph and its write gate, Store A, Store B and the guest store, indexes and query plans	the tape is the truth and every table is a view; a view without an access path costs CPU forever
2	The desk runtime (PDF)	agent loops and code as action, the four root desks and their authority, Yaegi cells, desk workers, records, inboxes and wakes, management delegation (S2a, S2b), crash versus planned restart	desks talk only through addressed records, and a crash restart ends work
3	The gateway and providers (PDF)	credential custody, guest identity, model catalog, policy and pools, retries, circuit breakers and failover, timeouts, Jev typed judgements, the search plane and budget routing, metering and prompt caching	a chokepoint is only as good as what it counts and what it trusts
4	The computer lifecycle (PDF)	microVMs and KVM, vmctl and vmmanager, the computer state machine, snapshots and hibernation, admission and pressure reclaim, the memory and disk model, deploys, refreshes and LU guest updates, runtime alternatives	a computer is not a VM: it is a durable identity that a VM sometimes runs
5	Publication and Autopaper (PDF)	sources and schedules, content-addressed captures, intake labels (L2a) and backpressure, stories as a precision and recall problem, Texture as sole writer, quote checking and Unicode, signed immutable publication to the Autopaper object graph, the front page	a pipeline of single-purpose authorities, each idempotent and each fact with one writer
6	Reasoning about trade-offs: a field guide (PDF)	failure modes, deadline budgets, correlation and races, idempotency, single state authority, fail closed versus fail open, blast radius, defence in depth, crash-only design, backpressure, observability, each with a real Choir incident; a one-page pre-flight card	most failures are not crashes, and every principle has a dated Choir receipt

Each document is 10,000 to 12,500 words: 19 to 22 printed pages in the dense A4 layout of the PDFs (21, 20, 19, 22, 21 and 20 pages), or about an hour's reading. Each ends with a glossary, five self-check questions with answer sketches, and the three files most worth reading next.

Two corrections the series makes to common shorthand (both code):

- **Store A is not "the guest store".** Store A is the host's platform control Dolt (event heads, idempotency, lifecycle, routes; port 13306). Store B is the host's corpus Dolt behind corpusd: captures and publications, the Autopaper object graph (port 13307). Each guest has its own embedded Dolt, which materializes its tape and has no letter (document 1, section 6).
- **Code still says "universal wire" and "sourcecycle".** Those are retired product names; the code rename is a TODO (Autopaper design §9). Read them as Autopaper (document 5).

How the parts fit

```
the world: feeds, web pages, model and search providers
      ^                                  |
      | (3) gateway: custody, routing,   | (5) sources, captures
      |     breakers, search, metering   v
+------------------------------ host: Node B ----------------------------------+
|  gateway :8084 ---- provider keys, Jev, search plane                  (3)  |
|  vmctl :8083 + vmmanager ---- Firecracker VMMs, snapshots, reclaim    (4)  |
|  corpusd :8086 ---- Store A :13306  platform control, event heads     (1)  |
|                \--- Store B :13307  corpus: captures, publications  (1,5)  |
|                        |                                                    |
|                        v  front page reads publications from Store B  (5)  |
+-------------------------------------------------------------------------+
      |  KVM boundary (4, and the isolation review)
      v
+-------------------- one persistent computer (a guest VM) ---------------------+
|  desks: Texture | management | engineering | research               (2)  |
|     each turn runs Go cells in Yaegi workers; desks talk by records        |
|  tape: append-only, hash-chained events; head CAS'd in Store A         (1)  |
|  embedded Dolt: the object graph, a materialization of the tape        (1)  |
|  Texture: sole writer of canonical documents; publishes to Store B   (2,5)  |
+-------------------------------------------------------------------------+

(6) the field guide is the lens for every arrow: what fails, who owns
    the fact, what a retry does, how far a breach reaches.
```

The numbers in brackets name the document that teaches that part.

Suggested reading order

The default path: skim 6, then 1 to 5 in order, then 6 in full.

1. **Twenty minutes first:** read section 12 of document 6, the pre-flight card. It lists the questions the other five documents answer for each subsystem, so reading it first tells you what to look for.
2. **Document 1, the ledger and the store.** Everything in Choir is a record on a tape or a row derived from one. Read this first: the others assume "tape", "head", "projection" and "object kind".
3. **Document 2, the desk runtime.** Who writes those records and how they move between desks. It builds on document 1's at-least-once delivery and idempotency.
4. **Document 3, the gateway and providers.** What the desks call out to, and how the platform contains a failing provider. This is the first document where availability and security pull against each other.
5. **Document 4, the computer lifecycle.** The machine underneath: VMs, snapshots, memory, deploys. Read the isolation security review right after it; document 4 points there instead of repeating it.
6. **Document 5, publication and Autopaper.** The whole system composed end to end, from a feed item to the front page. It is the best test of whether documents 1 to 4 stuck.
7. **Document 6, the field guide, in full.** Now each principle lands on incidents you have already met. Keep its pre-flight card beside you for every design review.

Shorter tracks, if one question is pressing:

if the question is	read	then
"why is the paper computer's CPU so high?"	1 (sections 7 and 8)	2 (section 5, records and inboxes)
"is the platform secure enough for strangers?"	4 (sections 1, 2, 8)	the isolation review, then 3 (section 2, guest identity) and 6 (sections 6 to 8)
"why did the paper stop or publish junk?"	5	2 (section 6, delegation), then 6 (section 10, backpressure)
"why did a deploy or restart hurt?"	4 (section 7)	2 (section 7), then 6 (section 9)
"what does a model call cost, and why?"	3 (sections 3 and 8)	the prompt cache analysis

How to study with these documents

You are learning at the limit, so the method matters more than the volume.

- **One document per sitting.** Each takes about an hour to read, and one to two hours with the code open. Do not read two in a row; the second will not stick.
- **Answer the five self-check questions before you read the answer sketches.** Write each answer in two or three sentences. Where you cannot, reread that section. Retrieval builds memory; rereading mostly does not.
- **Open the three files at the end of each document** and find the function each pointer names. Ten minutes in the real code is worth an hour of prose about it.
- **Trace one incident per document** from `docs/problems/` to its fix commit: `git log --oneline -- <file>` on the file the problem names, then `git show` on the fix. Ask the pre-flight card's questions of it.
- **Watch the evidence labels.** A claim marked *inference* is a hypothesis. The best exercise in the series is to pick one and find the trace or test that would confirm or refute it.

Where a document or comment disagrees with the code. The writers recorded these while grounding the series. All were read in code at 980ffaf4e, and each document names the paths. They are not fixes, only places to check:

- No production query uses Dolt's `AS OF`, branches or merges. History comes from the tape and the Texture revision chain (document 1).
- The embedded Dolt still runs on one connection behind an engine mutex, although the upstream race that required this is fixed (documents 1 and 6).
- The problem record crash restart leaves work open still says "not fixed", but the re-cast path it describes is gone and a crash closer exists (document 2).
- The front-importance timer reads the provider key file and calls OpenRouter directly, so the gateway is not the only holder of provider credentials (document 3).
- The gateway's guest-address check admits the three documentation address ranges in production. The isolation review flagged the same pattern in vmctl (document 3).
- Comments in `nix/autoputer-vm.nix` and `internal/vmmanager/manager.go` say KSM deduplicates guest pages, but the density report measured `pages_sharing=0` (document 4).
- Pressure reclaim can hibernate up to five computers in one pass from a single memory sample (document 4).
- Publication manifests are signed by corpusd's own key in `internal/platform/signing.go`, not by `internal/receiptsigner`, which signs guest execution receipts (document 5).
- No desk prompt says to treat captured text as data rather than as instructions (document 5).

What to learn next

The series teaches Choir's version of each idea. These are the sources that teach the ideas themselves, ordered by how much each one will pay back in this codebase in the next month. All are **upstream**; none is required to read the series.

Data and logs (pairs with document 1, then 5).

1. Martin Kleppmann, *Designing Data-Intensive Applications* (O'Reilly, 2017), chapters 3 (storage and retrieval), 5 (replication), 7 (transactions), 11 (stream processing). The single best book for the tape, the store and the projection pattern.
2. Jay Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction" (LinkedIn Engineering, 2013). Short; the argument for an append-only log as the source of truth.
3. Markus Winand, *Use The Index, Luke* (use-the-index-luke.com). How B-tree and secondary indexes turn into query plans. Read it with `EXPLAIN` open on a Dolt database.
4. DoltHub's documentation on prolly trees, the storage layout and `AS OF` (docs.dolthub.com, "Architecture" section). How Dolt differs from the InnoDB B-tree you met in Winand.
5. Alex Petrov, *Database Internals* (O'Reilly, 2019), part I. For when the query planner surprises you.

Agents and runtimes (pairs with document 2).

1. Joe Armstrong, *Making reliable distributed systems in the presence of software errors* (PhD thesis, 2003). The Erlang argument: processes, mailboxes, "let it crash", supervisors. The desks are actors; this is the theory of actors that survive failure.
2. George Candea and Armando Fox, "Crash-Only Software" (HotOS 2003). Four pages. The idea behind the crash-versus-planned-restart rule.
3. Xingyao Wang et al., "Executable Code Actions Elicit Better LLM Agents" (CodeAct, ICML 2024), and Alex Zhang and Omar Khattab, "Recursive Language Models" (2025). The research lineage behind cells instead of tool calls.

Platform reliability (pairs with documents 3 and 4).

1. Michael Nygard, *Release It!* (2nd ed., Pragmatic, 2018), chapters on stability patterns: timeouts, circuit breakers, bulkheads, steady state, fail fast. The vocabulary of document 3.
2. Google, *Site Reliability Engineering* (O'Reilly, 2016, free online), chapters 21 (handling overload) and 22 (addressing cascading failures). Then the AWS Builders' Library articles "Timeouts, retries and backoff with jitter" and "Making retries safe with idempotent APIs".
3. Jeffrey Dean and Luiz André Barroso, "The Tail at Scale" (CACM 2013). Why p99 latency, not the average, decides what a user feels.
4. Alexandru Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications" (NSDI 2020). The design of the VMM every Choir computer runs in, including snapshots and the jailer.
5. The Linux man pages `namespaces(7)`, `cgroups(7)`, `capabilities(7)` and `seccomp(2)` (man7.org), then the kernel's cgroup v2 admin guide. The raw material of every isolation boundary in document 4.

Security (pairs with documents 4 and 6).

1. Adam Shostack, *Threat Modeling: Designing for Security* (Wiley, 2014), part II. How to draw trust boundaries and ask what crosses them, which is what the isolation review does.
2. Mark S. Miller, *Robust Composition* (PhD thesis, 2006), chapters 1 to 9. Capabilities and least authority: the theory behind "a desk holds only the authority its role needs".
3. OWASP, "Top 10 for Large Language Model Applications" (2025 edition), on prompt injection and insecure output handling. The class of failure behind instruction text published as story titles (document 5).

Text and retrieval (pairs with document 5).

1. Unicode Standard Annex #15, "Unicode Normalization Forms", and UAX #29, "Text Segmentation". What a quote check has to agree on before two strings can be "the same".
2. Christopher Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval* (Cambridge, 2008, free online), chapters 1 to 8 and 16 to 17. Precision, recall, ranking and clustering: the mathematics of intake, stories and the front page.

Classics to keep on the desk.

1. Butler Lampson, "Hints for Computer System Design" (1983). Forty years old and still the best short list of design heuristics.
2. Leslie Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System" (1978). Why "what happened first" is a design decision, not a fact.

Inside Choir, after this series. Read Choir doctrine (the normative architecture), computer ontology, standing questions (the pre-flight question set; every question is backed by a dated failure), agent product doctrine, and the state and vision report. Then pick one open problem in `docs/problems/` a week and trace it from the problem doc to the code to the fix commit: that is the fastest way to turn the series into judgement.